Lecture 38: Deep Learning (Part-3) — Practice Problems Suite

This problem suite covers PyTorch implementation mechanics, ANN regression, multi-class classification, preprocessing pipelines, model checkpointing, and evaluation metrics.

Problem 1: Regression vs Multi-Class Classification Architecture

Compare the output layer design and loss functions of an ANN trained for regression versus an ANN trained for multi-class classification ($C = 7$). Why does regression have 1 output neuron with no activation, while multi-class classification has C output neurons without an explicit Softmax in PyTorch?

Problem 2: Mathematical Necessity of Feature Standardization

Why is feature standardization (`StandardScaler`) critical when training neural networks with gradient descent? What happens to the loss surface geometry and weight updates when features have radically different scales (e.g., `Area` $\approx 500,000$ vs `Eccentricity` ≈ 0.7)?

Problem 3: Data Leakage in Feature Scaling

Explain why executing `scaler.fit_transform(X)` on the entire dataset prior to `train_test_split` constitutes data leakage. What is the correct standard operating procedure using `fit_transform` and `transform` ?

Problem 4: PyTorch Data Pipeline Mechanics

Explain the roles of `TensorDataset` and `DataLoader` . Why can't we simply pass the full dataset tensor directly to `model(X)` for large datasets? What is the computational advantage of mini-batch processing with $B = 32$?

Problem 5: Tensor Data Types (`torch.float32` vs `torch.long`)

In PyTorch, why must target labels y be converted to `torch.float32` for `nn.MSELoss()` , but strictly converted to `torch.long` (`int64`) for `nn.CrossEntropyLoss()` ? What runtime error occurs if you pass `torch.float32` targets to `CrossEntropyLoss` ?

Problem 6: Manual Parameter Count Verification

Derive the exact number of learnable parameters (weights and biases) for:

1. The regression network: Input $4 \rightarrow$ Linear(64) $\rightarrow$ Linear(32) $\rightarrow$ Linear(1).
2. The classification network: Input $34 \rightarrow$ Linear(64) $\rightarrow$ Linear(64) $\rightarrow$ Linear(7).

Problem 7: The Softmax & `nn.CrossEntropyLoss` Trap

Explain why applying `nn.Softmax()` at the output layer of a PyTorch classification model trained with `nn.CrossEntropyLoss()` is a severe bug. What is the Log-Sum-Exp trick, and how does PyTorch maintain numerical stability?

Problem 8: The Four Pillars of the PyTorch Optimization Step

Enumerate the 4 critical steps executed in each training iteration:

1. `optimizer.zero_grad()`
2. `outputs = model(inputs)`
3. `loss.backward()`
4. `optimizer.step()` What disastrous phenomenon happens if `optimizer.zero_grad()` is commented out?

Problem 9: `model.state_dict()` vs Full Model Serialization

What exactly is `model.state_dict()`? Contrast saving a model using `torch.save(model.state_dict(), path)` versus saving the full model object `torch.save(model, path)`. Why is saving the state dictionary universally recommended in production systems?

Problem 10: Model Checkpointing and Overfitting Mitigation

Explain the mathematical and practical rationale behind model checkpointing. Why do we track validation loss rather than training loss to identify the "best" model? Why do we initialize `best_val_loss = float('inf')`?

Problem 11: Evaluation Guardrails (`model.eval()` and `torch.no_grad()`)

Explain the distinct functions of `model.eval()` and `with torch.no_grad():`. Does `model.eval()` stop gradient computation? Does `torch.no_grad()` disable dropout?

Problem 12: Dynamic Batch Size Handling in Accuracy Calculation

During evaluation across test mini-batches, why is writing `total += yb.size(0)` correct, while writing `total += 32` introduces a subtle mathematical bug when the dataset size is 180?

Problem 13: Regression Metric Interpretation

In the Power Plant regression task, our trained ANN achieved a test MSE of 17.81 and an R^2 score of 0.937.

1. Compute the test RMSE and explain its physical meaning in the context of electrical power output (MW).
2. Interpret what $R^2 = 0.937$ means regarding total variance.

Problem 14: Argmax Equivalence over Logits vs Probabilities

In multi-class prediction, we compute predicted classes using `_, predicted = torch.max(outputs, 1)` on raw logits. Prove mathematically that applying $\arg\max$ directly on raw logits z_i produces the exact same predicted class as applying $\arg\max$ on Softmax probabilities $P(y = i|\mathbf{x})$.

Problem 15: Dimensionality Reduction Challenge (PCA + ANN)

Describe how you would integrate Principal Component Analysis (PCA) into the Date Fruit classification workflow. If 34 features are reduced to k principal components explaining 95% cumulative variance, outline the revised PyTorch architecture and pipeline.